Lab 4: VibeCoding - Chat History

Tool used: Claude (claude.ai)
Assigned repo: SETAPESU26/40_flappy-bird (Flappy Bird, Python/Pygame)
Student fork: https://github.com/Mohd-Sinan7/40_flappy-bird
Full conversation (share link): https://claude.ai/share/b3f1f0a3-1335-4720-a615-ee00ad87e176

This is a condensed transcript of the relevant part of the conversation. Student messages are quoted as sent; Claude's replies are summarised, and the code that resulted from each task is included in the appendix. The complete, unedited chat is at the share link above.

1. Understanding the assignment

Student:
[Uploaded: Lab_4_VibeCoding_Student_handout.pdf]

https://github.com/SETAPESU26/40_flappy-bird
this is the github link assigned to me and this is the lab4 instructions to do explain me what all to be done and lets complete the submission

Claude:
Read the handout and the repo README and summarised the lab: fork/clone the repo, record a 10-second "before" video, use prompts to fix the broken code and add the README features one by one with a separate commit per task, record a 10-second "after" video, push to the student's own repo (no PR to SETAPESU26), and upload videos, code and chat history to a Lab-4 folder.

The README lists four tasks: (1) refine collision detection, (2) Game Over screen showing the final score, (3) replay option with Easy/Medium/Hard difficulty or exit, (4) sound effects for flap, score and death.

Student:
lets do step by step

Claude:
Gave Step 1: fork the repo, clone the fork, run `pip install -r requirements.txt` and `python main.py` without changing any code.

Student:
lets proceed next step

Claude:
Gave Step 2: how to record the 10-second "before" video, what to show (the collision bug and the silent freeze on death), and recorder shortcuts.

2. Finding the bug in the repo

Student:
i mean what is the bug present in the current repo and what is exactly to be fixed

Claude:
Cloned the repo and read every file (`main.py`, `game/bird.py`, `game/pipe.py`, `game/game_engine.py`). **The bug:** pipe collision only tested the single centre point of the bird, although

the bird is drawn as a circle of radius 15, so the bird could overlap a pipe edge and survive:

```
if pipe.top_rect().collidepoint(self.bird.center()) or pipe.bottom_rect().collidepoint(self.bird.center()):
    self.game_over = True
```

Claude:
Also noted: the Game Over state only did a console `print`, the game could not be restarted, difficulty was fixed (`pipe_speed = 4`, gap 150), and there was no audio.

3. Fixing and adding features, one task at a time

Student:
this is done now lets fix everything

Claude:
Implemented the four tasks locally as four separate commits and tested each one headlessly before moving on (all passed):

Task 1 - Fix collision detection

Prompt used: "Here's bird.py, pipe.py and game_engine.py. The bird sometimes passes through pipe edges. Find the cause in the collision logic and fix it using the bird's full circular body."

Added `Bird.collides_with_rect(rect)`: finds the point on the rectangle closest to the bird's centre and checks whether it lies within one radius. `update()` now calls it for the top and bottom pipe rectangles. Test: a pipe edge 10 px from the bird's centre now ends the game; the old centre-point check did not detect it; flying through the gap is still safe.

Task 2 - Game Over screen

Prompt used: "Add a Game Over state that triggers on ground, ceiling or pipe collision. Draw the final score on screen and wait for input instead of printing to the console."

Replaced the console print with a dark overlay showing GAME OVER and the final score. Flapping is ignored while game over, and Q/ESC quits. Test: ground and ceiling both trigger it, flap is ignored, ESC posts a QUIT event, screenshot checked.

Task 3 - Replay with difficulty

Prompt used: "After Game Over, show a menu: Easy / Medium / Hard (changes pipe speed and gap size) or Quit. Selecting a difficulty restarts the game with all state reset."

Added a `DIFFICULTIES` table (Easy speed 3 / gap 200, Medium 4 / 150, Hard 6 / 120), a `reset(difficulty)` method, and a menu with keys 1/2/3, clickable buttons, and Q/ESC to exit. Pipe spacing is kept constant across difficulties by deriving the spawn interval from the speed. Test: keyboard and mouse selection both restart correctly, stray clicks are ignored, and a 5000-frame random-play soak test ran on all three difficulties without errors.

Task 4 - Sound effects

Prompt used: "Add sound effects for flap, score and death using pygame.mixer. Generate the sounds in code, and fail gracefully if there is no audio device."

New file `game/sound.py` synthesises the three sounds with the standard library (no audio files needed). The engine plays them on flap, on passing a pipe, and once on death. If the mixer cannot start, every call becomes a silent no-op. Test: sound calls fire exactly when expected, death plays once per game, the fallback works, and the full `main.py` loop starts and exits cleanly.

The patches for all four tasks are in the appendix.

4. Applying the fixes to the student's fork

Student:
C:\Users\DELL\40_flappy-bird>bash /path/to/lab4_flappy_bird/apply_patches.sh ...
'bash' is not recognized as an internal or external command

Claude:
Windows Command Prompt has no `bash`, so the script was replaced by plain `git apply` / `git add` / `git commit` commands for each task, using the full path to the patch files. After the Task 1 commit also picked up compiled `__pycache__` files, the commit was amended to remove them and `__pycache__/` was added to `.git/info/exclude`.

Student:
[git log output]
ec965f8 Task 4: add sound effects (flap, score, die)
b511ecb Task 3: add replay option with Easy/Medium/Hard difficulty or exit
f6d5e9b Task 2: add Game Over screen with final score
cfbf9d2 Task 1: fix collision detection (circle vs rect)

Claude:
Confirmed four clean, separate commits on top of the original two and a clean working tree. Then gave the checklist to test the finished game and record the "after" video.

Student:
[git push origin main output]
a605733..ec965f8 main -> main

Claude:
Confirmed the four task commits were pushed to the student's own fork (Mohd-Sinan7/40_flappy-bird), not to SETAPESU26.

5. Organising the submission

Student:
i want my lab4 folder also to be like this
[screenshot of the repo: Lab-1, Lab-2, Lab-3 each containing one folder per student]

Claude:
Confirmed the layout `Lab-4/Mohammed_Sinan_PES1UG24AM165/` (containing `flappy_bird/`, `before.mp4`, `after.mp4`) matches Lab-3. Spotted a leftover duplicate copy at the repo root from an earlier

push, and gave commands to record its removal without touching the Lab-4 files. The chat history (this document) and `chat_link.txt` are added to the same folder.

Appendix: code changes per task (git patches)

Task 1 - Fix collision detection (commit: Task 1)

```
diff --git a/game/bird.py b/game/bird.py
index 72fa131..c942244 100644
--- a/game/bird.py
+++ b/game/bird.py
@@ -21,3 +21,15 @@ class Bird:

     def rect(self):
         return pygame.Rect(self.x - self.radius, self.y - self.radius, self.radius * 2, self.radius * 2)
+
+    def collides_with_rect(self, rect):
+        """Circle-vs-rectangle collision using the bird's full radius.
+
+        Finds the point on the rectangle closest to the bird's center and
+        checks whether it lies within one radius of the center.
+        """
+        closest_x = max(rect.left, min(self.x, rect.right))
+        closest_y = max(rect.top, min(self.y, rect.bottom))
+        dx = self.x - closest_x
+        dy = self.y - closest_y
+        return dx * dx + dy * dy <= self.radius * self.radius
diff --git a/game/game_engine.py b/game/game_engine.py
index 28d8719..926ed51 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -52,12 +52,9 @@ class GameEngine:
         for pipe in self.pipes:
             pipe.move()

-            # NOTE: collision only checks the bird's single center point
-            # against the pipe rects, rather than the bird's full rect.
-            # At higher pipe speeds the bird can visually clip a pipe's
-            # edge for a frame or two without this ever registering a
-            # hit. See Task 1 in the README.
-            if pipe.top_rect().collidepoint(self.bird.center()) or
      pipe.bottom_rect().collidepoint(self.bird.center()):
+            # Collision uses the bird's full circular body (not just its
+            # center point), so clipping a pipe edge is always detected.
+            if self.bird.collides_with_rect(pipe.top_rect()) or
      self.bird.collides_with_rect(pipe.bottom_rect()):
                 self.game_over = True

             if not pipe.scored and pipe.x + pipe.width < self.bird.x:
```

Task 2 - Game Over screen (commit: Task 2)

```
diff --git a/game/game_engine.py b/game/game_engine.py
index 926ed51..0469791 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -20,9 +20,17 @@ class GameEngine:

         self.score = 0
         self.font = pygame.font.SysFont("Arial", 30)
+        self.big_font = pygame.font.SysFont("Arial", 56, bold=True)
+        self.small_font = pygame.font.SysFont("Arial", 24)
         self.game_over = False

     def handle_event(self, event):
+        if self.game_over:
+            # Game Over screen: wait for input. ESC / Q requests quit.
+            if event.type == pygame.KEYDOWN and event.key in (pygame.K_ESCAPE, pygame.K_q):
+                pygame.event.post(pygame.event.Event(pygame.QUIT))
+            return
+
         # Flap is edge-triggered (KEYDOWN / MOUSEBUTTONDOWN), not held.
         if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
             self.bird.flap()
@@ -73,7 +81,19 @@ class GameEngine:
         score_text = self.font.render(f"Score: {self.score}", True, WHITE)
         screen.blit(score_text, (10, 10))
```

```
-        if self.game_over and not getattr(self, "_game_over_logged", False):
-            # NOTE: no proper game-over screen yet - see Task 2 in the README.
-            print("Game over! Final score:", self.score)
-            self._game_over_logged = True
+        if self.game_over:
+            self._render_game_over(screen)
+
+    def _render_game_over(self, screen):
+        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
+        overlay.fill((0, 0, 0, 160))
+        screen.blit(overlay, (0, 0))
+
+        lines = [
+            (self.big_font, "GAME OVER", (255, 80, 80), self.height // 2 - 80),
+            (self.font, f"Final Score: {self.score}", WHITE, self.height // 2),
+            (self.small_font, "Press ESC or Q to quit", WHITE, self.height // 2 + 60),
+        ]
+        for font, text, color, y in lines:
+            surf = font.render(text, True, color)
+            screen.blit(surf, surf.get_rect(center=(self.width // 2, y)))
```

Task 3 - Replay with difficulty (commit: Task 3)

```
diff --git a/game/game_engine.py b/game/game_engine.py
index 0469791..cd2c234 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -7,28 +7,58 @@ from .pipe import Pipe
 WHITE = (255, 255, 255)
 GREEN = (0, 150, 0)

+# Difficulty presets: pipe speed (px/frame) and gap height (px).
+DIFFICULTIES = {
+    "Easy":   {"speed": 3, "gap": 200},
+    "Medium": {"speed": 4, "gap": 150},
+    "Hard":   {"speed": 6, "gap": 120},
+}
+DEFAULT_DIFFICULTY = "Medium"
+PIPE_SPACING = 360  # horizontal distance between pipes, kept constant across difficulties
+
+
 class GameEngine:
     def __init__(self, width, height):
         self.width = width
         self.height = height

-        self.bird = Bird(width // 4, height // 2)
-        self.pipe_speed = 4
-        self.pipe_interval = 90  # frames between pipe spawns
-        self._spawn_timer = 0
-        self.pipes = [Pipe(width + 100, height, speed=self.pipe_speed)]
-
-        self.score = 0
         self.font = pygame.font.SysFont("Arial", 30)
         self.big_font = pygame.font.SysFont("Arial", 56, bold=True)
         self.small_font = pygame.font.SysFont("Arial", 24)
+
+        self._build_menu_buttons()
+        self.reset(DEFAULT_DIFFICULTY)
+
+    # ------------------------------------------------------------------ setup
+    def reset(self, difficulty):
+        """Start a fresh game with the chosen difficulty."""
+        self.difficulty = difficulty
+        settings = DIFFICULTIES[difficulty]
+        self.pipe_speed = settings["speed"]
+        self.pipe_gap = settings["gap"]
+        self.pipe_interval = round(PIPE_SPACING / self.pipe_speed)  # frames between spawns
+
+        self.bird = Bird(self.width // 4, self.height // 2)
+        self._spawn_timer = 0
+        self.pipes = [Pipe(self.width + 100, self.height, gap=self.pipe_gap, speed=self.pipe_speed)]
+
+        self.score = 0
         self.game_over = False

+    def _build_menu_buttons(self):
```

```
+        """Clickable difficulty buttons shown on the Game Over screen."""
+        btn_w, btn_h, spacing = 260, 48, 14
+        top = self.height // 2 + 40
+        self.menu_buttons = []
+        for i, name in enumerate(DIFFICULTIES):
+            rect = pygame.Rect(0, 0, btn_w, btn_h)
+            rect.center = (self.width // 2, top + i * (btn_h + spacing) + btn_h // 2)
+            self.menu_buttons.append((name, rect))
+
+    # ------------------------------------------------------------------ input
     def handle_event(self, event):
         if self.game_over:
-            # Game Over screen: wait for input. ESC / Q requests quit.
-            if event.type == pygame.KEYDOWN and event.key in (pygame.K_ESCAPE, pygame.K_q):
-                pygame.event.post(pygame.event.Event(pygame.QUIT))
+            self._handle_game_over_event(event)
             return

         # Flap is edge-triggered (KEYDOWN / MOUSEBUTTONDOWN), not held.
@@ -37,11 +67,28 @@ class GameEngine:
         if event.type == pygame.MOUSEBUTTONDOWN:
             self.bird.flap()

+    def _handle_game_over_event(self, event):
+        if event.type == pygame.KEYDOWN:
+            if event.key in (pygame.K_ESCAPE, pygame.K_q):
+                pygame.event.post(pygame.event.Event(pygame.QUIT))
+            elif event.key == pygame.K_1:
+                self.reset("Easy")
+            elif event.key == pygame.K_2:
+                self.reset("Medium")
+            elif event.key == pygame.K_3:
+                self.reset("Hard")
+        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
+            for name, rect in self.menu_buttons:
+                if rect.collidepoint(event.pos):
+                    self.reset(name)
+                    break
+
     def handle_input(self):
         # Reserved for continuously-held-key input; flapping is handled
         # in handle_event instead, so there's nothing to poll here.
         pass

+    # ---------------------------------------------------------------- update
     def update(self):
         if self.game_over:
             return
@@ -55,7 +102,7 @@ class GameEngine:
         self._spawn_timer += 1
         if self._spawn_timer >= self.pipe_interval:
             self._spawn_timer = 0
-            self.pipes.append(Pipe(self.width, self.height, speed=self.pipe_speed))
+            self.pipes.append(Pipe(self.width, self.height, gap=self.pipe_gap, speed=self.pipe_speed))

         for pipe in self.pipes:
             pipe.move()
@@ -71,6 +118,7 @@ class GameEngine:

         self.pipes = [p for p in self.pipes if not p.off_screen()]

+    # ---------------------------------------------------------------- render
     def render(self, screen):
         for pipe in self.pipes:
             pygame.draw.rect(screen, GREEN, pipe.top_rect())
@@ -80,6 +128,8 @@ class GameEngine:

         score_text = self.font.render(f"Score: {self.score}", True, WHITE)
         screen.blit(score_text, (10, 10))
+        mode_text = self.small_font.render(self.difficulty, True, WHITE)
+        screen.blit(mode_text, mode_text.get_rect(topright=(self.width - 10, 12)))

         if self.game_over:
             self._render_game_over(screen)
@@ -90,10 +140,21 @@ class GameEngine:
         screen.blit(overlay, (0, 0))

         lines = [
```

```
-            (self.big_font, "GAME OVER", (255, 80, 80), self.height // 2 - 80),
-            (self.font, f"Final Score: {self.score}", WHITE, self.height // 2),
-            (self.small_font, "Press ESC or Q to quit", WHITE, self.height // 2 + 60),
+            (self.big_font, "GAME OVER", (255, 80, 80), self.height // 2 - 130),
+            (self.font, f"Final Score: {self.score}", WHITE, self.height // 2 - 70),
+            (self.small_font, "Play again - choose difficulty:", WHITE, self.height // 2 + 10),
         ]
         for font, text, color, y in lines:
             surf = font.render(text, True, color)
             screen.blit(surf, surf.get_rect(center=(self.width // 2, y)))
+
+        mouse_pos = pygame.mouse.get_pos()
+        for i, (name, rect) in enumerate(self.menu_buttons, start=1):
+            hovered = rect.collidepoint(mouse_pos)
+            pygame.draw.rect(screen, (60, 170, 60) if hovered else (40, 120, 40), rect, border_radius=8)
+            pygame.draw.rect(screen, WHITE, rect, width=2, border_radius=8)
+            label = self.small_font.render(f"[{i}] {name}", True, WHITE)
+            screen.blit(label, label.get_rect(center=rect.center))
+
+        hint = self.small_font.render("Press ESC or Q to exit", True, WHITE)
+        screen.blit(hint, hint.get_rect(center=(self.width // 2, self.height - 60)))
```

Task 4 - Sound effects (commit: Task 4)

```
diff --git a/game/game_engine.py b/game/game_engine.py
index cd2c234..296da72 100644
--- a/game/game_engine.py
+++ b/game/game_engine.py
@@ -1,6 +1,7 @@
 import pygame
 from .bird import Bird
 from .pipe import Pipe
+from .sound import SoundManager

 # Game Engine

@@ -26,6 +27,7 @@ class GameEngine:
         self.big_font = pygame.font.SysFont("Arial", 56, bold=True)
         self.small_font = pygame.font.SysFont("Arial", 24)

+        self.sounds = SoundManager()
         self._build_menu_buttons()
         self.reset(DEFAULT_DIFFICULTY)

@@ -63,9 +65,19 @@ class GameEngine:

         # Flap is edge-triggered (KEYDOWN / MOUSEBUTTONDOWN), not held.
         if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
-            self.bird.flap()
+            self._flap()
         if event.type == pygame.MOUSEBUTTONDOWN:
-            self.bird.flap()
+            self._flap()
+
+    def _flap(self):
+        self.bird.flap()
+        self.sounds.flap()
+
+    def _end_game(self):
+        """Mark the game as over (and play the death sound exactly once)."""
+        if not self.game_over:
+            self.game_over = True
+            self.sounds.die()

     def _handle_game_over_event(self, event):
         if event.type == pygame.KEYDOWN:
@@ -96,7 +108,7 @@ class GameEngine:
         self.bird.update()

         if self.bird.y - self.bird.radius <= 0 or self.bird.y + self.bird.radius >= self.height:
-            self.game_over = True
+            self._end_game()
             return

         self._spawn_timer += 1
@@ -110,11 +122,12 @@ class GameEngine:
             # Collision uses the bird's full circular body (not just its
```

```
             # center point), so clipping a pipe edge is always detected.
             if self.bird.collides_with_rect(pipe.top_rect()) or
      self.bird.collides_with_rect(pipe.bottom_rect()):
-                self.game_over = True
+                self._end_game()

             if not pipe.scored and pipe.x + pipe.width < self.bird.x:
                 pipe.scored = True
                 self.score += 1
+                self.sounds.score()

         self.pipes = [p for p in self.pipes if not p.off_screen()]

diff --git a/game/sound.py b/game/sound.py
new file mode 100644
index 0000000..73b9669
--- /dev/null
+++ b/game/sound.py
@@ -0,0 +1,57 @@
+import math
+import array
+import pygame
+
+
+class SoundManager:
+    """Generates and plays the game's sound effects.
+
+    The sounds are synthesized in code (no asset files needed). If audio
+    cannot be initialised (no sound device, etc.) every method becomes a
+    silent no-op so the game still runs.
+    """
+
+    def __init__(self):
+        self.enabled = False
+        self.flap_sound = self.score_sound = self.die_sound = None
+        try:
+            if not pygame.mixer.get_init():
+                pygame.mixer.init(22050, -16, 1)
+            self.freq, fmt, self.channels = pygame.mixer.get_init()
+            if fmt != -16:  # we only synthesize signed 16-bit samples
+                return
+            self.flap_sound = self._make([(400, 800, 0.08)], volume=0.35)
+            self.score_sound = self._make([(880, 880, 0.07), (1320, 1320, 0.12)], volume=0.35)
+            self.die_sound = self._make([(500, 120, 0.50)], volume=0.4)
+            self.enabled = True
+        except (pygame.error, NotImplementedError):
+            self.enabled = False
+
+    def _make(self, segments, volume=0.4):
+        """Build a pygame Sound from (start_hz, end_hz, seconds) tone segments."""
+        samples = array.array("h")
+        phase = 0.0
+        for start_hz, end_hz, seconds in segments:
+            n = int(self.freq * seconds)
+            for i in range(n):
+                t = i / n
+                hz = start_hz + (end_hz - start_hz) * t
+                phase += 2 * math.pi * hz / self.freq
+                envelope = 1.0 - t  # fade out to avoid clicks
+                value = int(32767 * volume * envelope * math.sin(phase))
+                for _ in range(self.channels):
+                    samples.append(value)
+        return pygame.mixer.Sound(buffer=samples.tobytes())
+
+    def _play(self, sound):
+        if self.enabled and sound is not None:
+            sound.play()
+
+    def flap(self):
+        self._play(self.flap_sound)
+
+    def score(self):
+        self._play(self.score_sound)
+
+    def die(self):
+        self._play(self.die_sound)
diff --git a/main.py b/main.py
index 77021b1..4f72838 100644
```

```
--- a/main.py
+++ b/main.py
@@ -2,6 +2,7 @@ import pygame
 from game.game_engine import GameEngine

 # Initialize pygame/Start application
+pygame.mixer.pre_init(22050, -16, 1, 512)  # low-latency mono audio for sound effects
 pygame.init()

 # Screen dimensions
```